Java Fundamentals

Student Workbook 1c – Introduction to Java

Version 7.1 Y

Table of Contents

Module 1 Introduction to Programming **5**
Section 1–1 Thinking About Programming 6
Programming 7
Programming 8
Section 1–2 Processing Programs 9
Binary, Bits and Processing 10
Understanding Binary 11
Binary, Bits and Processing 12
Exercise – Your Lucky Number in Binary 13
File sizes and Processing Speeds 14
Bytes and Storage 15
Exercise – Decimal to Binary 16
Exercise – Binary to Decimal 17
Section 1–3 Learning to Be Precise 18
Learning to Be Precise 19
Examples: Program the Monkey 20
Exercises: Program the Monkey 21
Expanded Command Vocabulary 22
Examples: Monkey with a Basket 23
Exercises – Monkey with a Basket 24
Section 1–4 Dealing with Ambiguity 27
Dealing with Ambiguity 28
Trying to Be Exact 29
Complex Problems 32
Exercises – Brush Your Teeth 33
Module 2 Introducing Java **34**
Section 2–1 Overview of Java 35
Java 36
Java Bytecode and the Java Virtual Machine (JVM) 37
Java Runtime Environment (JRE) 38
Java Developer Kit (JDK) 39
Compiling Java 40
Module 3 Basic Java Syntax **41**
Section 3–1 Basic Java Syntax 42
Java Syntax 43
Javadoc 44
Example: Code Written with Javadoc in Mind 45
Data Types 46
Variables 47
Data Types: Numbers 48
Data Types: Characters 49
Types of Variables 50
Read-only Variables 51
Default Values 52
Naming Conventions and Consistency 53
Exercise – Variables 54
Section 3–2 Operators and Expressions 55
String Concatenation 56
Mathematical Operators and Expressions 57
Pre/Post - Increment and Decrement 59

Literals Are Typed ... 60
Widening Issues ... 62
Narrowing Issues ... 63
Type Casting ... 65
Java's `Math` Class ... 66
Example: Working with the `Math` Class ... 67
Assignment Operators ... 68
Exercises – Operators ... 69
Section 3–3 Writing to the Screen and Reading from the Keyboard ... 73
Writing Text to the Screen ... 74
Formatting Output ... 75
Java Format Specifiers ... 76
Reading Input with Scanner ... 77
Read a Whole Line of Text ... 78
Read Individual Values ... 79
Example: A Calculator ... 80
Mixed Input Types ... 81
Buffered Input ... 82
Line Separators and Numeric Input ... 83
Line Separators and nextLine() ... 84
Dealing With Line Separators ... 85
Consuming a Line Separator ... 86
Exercises – Basic Calculator ... 87
Learning Effectively ... 89
AI and Learning Effectively ... 91
AI Ethics ... 92
AI Ethics ... 93
Example: Using AI Effectively ... 94
Module 4 Conditionals and Methods ... 95
Section 4–1 Conditionals ... 96
Conditionals ... 97
`if` Statement ... 98
`if / else` ... 99
Comparing Strings ... 101
Example: Better Calculator? ... 102
Exercises – Sandwich Shop ... 103
Section 4–2 Static Methods ... 105
Static Methods ... 106
Declaring a static Method ... 107
Method Responses ... 108
Passing Data to Methods ... 109
Returning Data from Methods ... 111
Using a `Scanner` in Multiple Methods ... 112
Example: Re-declaring the Scanner ... 113
Example: Passing the Scanner ... 115
Exercises – Calculator with Methods ... 117
Section 4–3 The `switch` Statement ... 120
`switch` Statement ... 121
`switch` Example ... 122
The break statement ... 123
Exercises – Rental Car Company ... 124
Conditional Operator ... 126
Module 5 Continued Learning ... 127
Section 5–1 Debugging Project ... 128
Debugging Project ... 129

Section 5–2 Practice Repository 130
Practice Problems 131

Module 1

Introduction to Programming

Section 1–1

Thinking About Programming

Programming

- **Programming is the process of telling a computer what to do**
  - When we program, we write instructions – step by step – in a way the computer understands

- **This seems incredibly basic, yet there is a difference between instructing computers and instructing a person**
  - Let's say we are telling a friend how to make a peanut butter & jelly sandwich – Our friends can typically fill in the gaps

- **When we tell a computer what to do, there is no room for ambiguity**
  - Every single step must be explicitly stated
  - Every detail must be specified
  - Computers do exactly what we tell them to do

- **This need for precision is why we need programming languages**

Programming

- **There are many programming languages to choose from**
  - To ensure we provide computers with these precise instructions, we use programming languages
  - There are many programming languages, each have strengths for different problems

- **This week, we will begin learning Java**
  - Java is one of the most widely-used programming languages
    - It is used by more than 9 million developers
    - Is used for everything from Android Applications to large-scale financial systems
    - Provides the fundamental concepts that transfer to almost any other programming language

- **Along the way, we will use other languages and data formats:**
  - Such as MySQL, XML, JSON, and Bash

Section 1–2

Processing Programs

Binary, Bits and Processing

- **What does a computer actually process?**
  - When you write a program in Java, the computer doesn't read "Java"
  - Everything, gets translated into binary: sequences of 1s and 0s
  - This is the only language computers understand

- **How are these programs translated?**
  - Our Java code will go through a process called compilation
    - We will talk more later on about how the compiler will transform our code into binary instructions for the computer to process
  - Each instruction, each piece of data, will become a pattern of 1s and 0s

- **This makes precision absolutely critical**
  - A single misplaced bit, a 0 that should be a 1, could be catastrophic
    - A 64 changes to a 65
    - Valid instructions get turned into gibberish
    - Entire programs will crash

Understanding Binary

- **A bit is a single binary digit: either 1 or 0**
  - You can think of this like a light switch: on or off, electricity *is* flowing or it *isn't*
  - Every piece of information in a computer is built from billions of these switches

- **How we count in everyday life (Base 10)**
  - When we count as humans, we use ten different digits:
    - 1, 2, 3, 4, 5, 6, 7, 8, 9
  - When we run out of digits, we add another place
    - 9 -> 10 (one ten, zero ones)
    - 99 -> 100 (one hundred, zero tens, zero ones)
  - This system is called "decimal numbers" (coming from the Latin word decem, meaning ten)
  - We call this "Base 10" because we use 10 different digits

- **How computers count (Base 2)**
  - Computers only use two different digits: 0 and 1
  - When computers run out of digits, they add another place:
    - 1 -> 10 (one two, zero ones)
    - 11 -> 100 (one four, zero twos, zero ones)
  - Each binary digit is called a **bit** (short for "binary digit")

Binary, Bits and Processing

- **Let's break down how binary works**
  - As we've seen with **decimal** numbers, each position represents a power of **10** (ones, tens, hundreds)

	128	64	32	16	8	4	2	1
Binary:	1	0	1	1	0	1	0	1

∗ This binary number (10110101) equals: 128 + 32 + 16 + 4 + 1 = **181 decimal**

- **In Binary, each position represents a power of 2**

128	64	32	16	8	4	2	1

Exercise – Your Lucky Number in Binary

- **in Base 2?**

```
___  ___  ___  ___  ___  ___  ___  ___        digits are 0 - 1
128   64   32   16    8    4    2    1
```

- **in Base 10 (this should be easy)?**

```
___  ___  ___        digits are 0 - 9
100   10    1
```

- **How many bits did you need to represent each?**

File sizes and Processing Speeds

- **How much data can a computer handle?**

- **32-bit vs 64-bit processors:**
  - When you see computers marketed as "32-bit" or "64-bit," this refers to how much data the processor can work with at one time

- **A 32-bit computer**
  - Can process 32 bits of information in a single operation
  - The largest signed whole number it can store is **2,147,483,647**
  - In binary: `01111111111111111111111111111111`

    ∗ Note: The first bit (0) indicates it's positive. That's why it starts with 0, not 1

- **A 64-bit computer**
  - Can process 64 bits of information in a single operation
  - The largest signed whole number it can store is **9,223,372,036,854,775,807**
  - That's over 9 *quintillion*

- **This is why modern computers are 64-bit: they can handle much larger numbers and more memory**

Bytes and Storage

- **A <u>bit</u> is a single binary digit: 1 or 0**

- **A <u>byte</u> is a group of 8 bits**
  - The smallest unit of memory you can access is one byte (8 bits)
  - This is enough to represent 256 different values, which is perfect for storing a single character, a small number, or a color value

- **Programmers measure data in increasingly larger units:**

UNIT	SIZE	BYTES
1 Bit	Single 0 or 1 binary digit	--
1 Byte	8 bits	1 byte
1 Kilobyte (KB)	1024 bytes	1024 bytes
1 Megabyte (MB)	1024 KB	1,048,576 bytes
1 Gigabyte (GB)	1024 MB	1,073,741,824 bytes
1 Terabyte (TB)	1024 GB	1,099,511,627,776 bytes

- **Why 1024 instead of 1000?**
  - In the metric system, "kilo" means 1000. But in computing, we use 1024 because that's 2^10, which aligns with how binary works.

Exercise – Decimal to Binary

- **How would you write the following Base10 (Decimal) numbers in Binary?**
  - 15

    __ __ __ __ __ __ __ __
  - 32

    __ __ __ __ __ __ __ __
  - 117

    __ __ __ __ __ __ __ __
  - 946

    __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __

Exercise – Binary to Decimal

- **How would you write the following Binary numbers in Decimal (Base10)**
  - 00000100

    __ __ __ __ __ __ __ __
  - 00001101

    __ __ __ __ __ __ __ __
  - 00110101

    __ __ __ __ __ __ __ __
  - 01010101 10101010

    __ __ __ __ __ __ __ __ __ __ __ __ __ __ __ __

Section 1–3

Learning to Be Precise

Learning to Be Precise

- **To demonstrate using precise instructions, we'll spend a little time on a paper/pen coding exercise**

- **In this first phase, we will make a monkey move to a banana and eat it using a set of specific commands**
  - Note: the ideas for this exercise came from `codemonkey.com`[1]

- **Right now, your monkey understands 3 commands**

`Step` *number*
where *number* is the number of squares to move

Example: `Step 5`

`Turn` *direction*
where *direction* is `left` or `right`

Example: `Turn left`

`Eat banana`
This only works if the monkey is standing on an adjacent square to the banana *and* facing it.

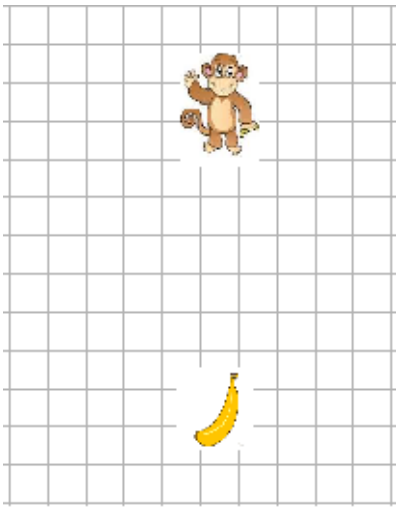
Example: `Eat banana`

[1] And before there was codemonkey.com, there was a really old programming language called "LOGO", in which you could move "turtles" to draw pictures

Examples: Program the Monkey

Example

Move the monkey to the banana and have him eat it.

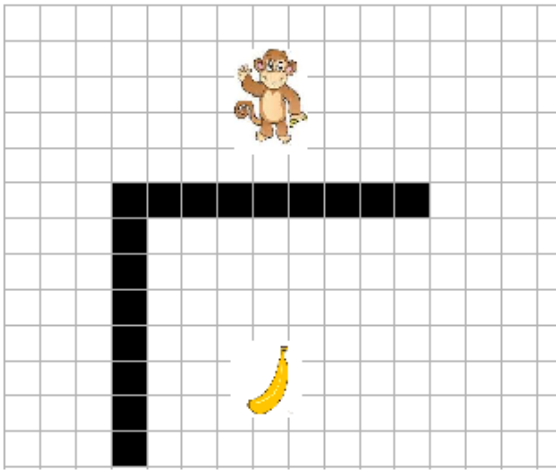


SOLUTION

```
Step 6
Eat banana
```

Example

Move the monkey to the banana and have him eat it. Avoid the barrier. Pay attention to the direction the monkey is facing.



SOLUTION

```
Turn left
Step 5
Turn right
Step 7
Turn right
Step 4
Eat banana
```

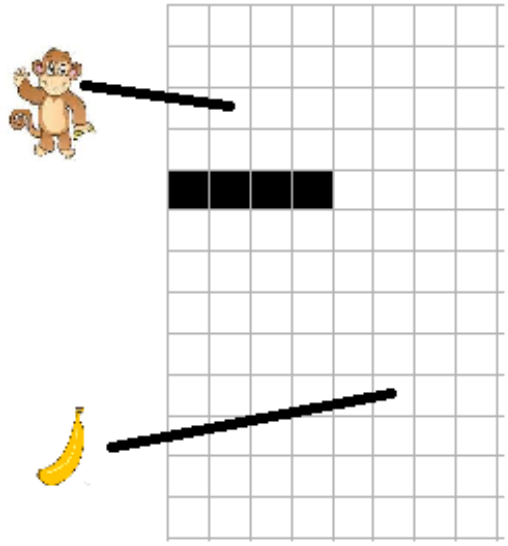
- **Note: Just like programming in real life, there is often more than one way solve a problem**

Exercises: Program the Monkey

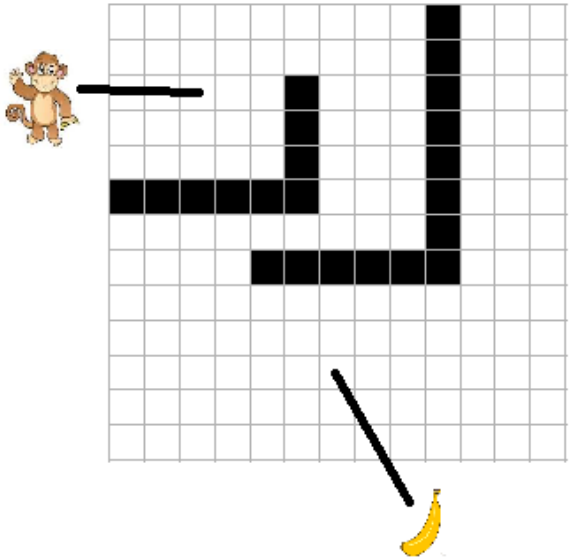
In the next few exercises, we will let you figure out the steps needed to "program your monkey" to move to and eat the banana. We are using lines to show you where the monkey and banana are so that the positions are very precise.

When you finish, talk it over in your group to see how others programmed their monkey.

EXERCISE 1



EXERCISE 2



Expanded Command Vocabulary

- **When you are learning a programming language, you start with just a few commands and learn to use them**
  - When you conquer them, you learn more commands and how to use them too!
  - Before long, you know a LOT about the programming language

- **Your monkey understands the commands we discussed before, plus some commands that can be used to pick up or drop an item**

`Step` *number*
where *number* is the number of squares to move

`Turn` *direction*
where *direction* is `left` or `right`

`Pickup` *item*
where *item* is `banana` or `basket`

When you pick up an item, you must be facing the item in an adjacent square.

Example: `Pickup basket`

`Drop` *item*
where *item* is `banana` or `basket`

When you drop an item, it stays in the square directly in front of the square you are standing in. NOTE: Only one item can reside on a square, however, if the banana is IN the basket that counts as one item.

Example: `Drop basket`

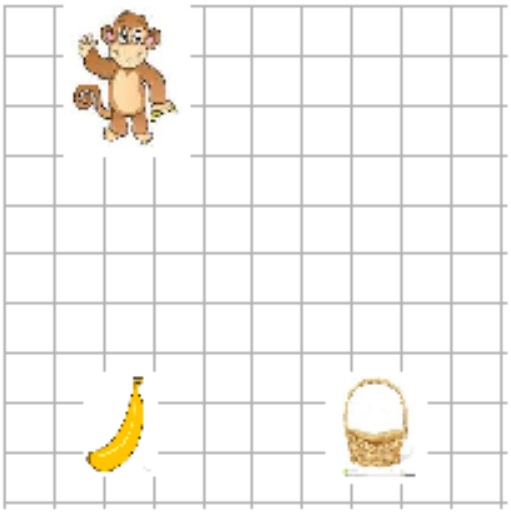
`Eat banana`

This only works if the monkey is standing on square adjacent to the banana directly facing it.

Examples: Monkey with a Basket

Example

Make the monkey drop the banana into the basket.



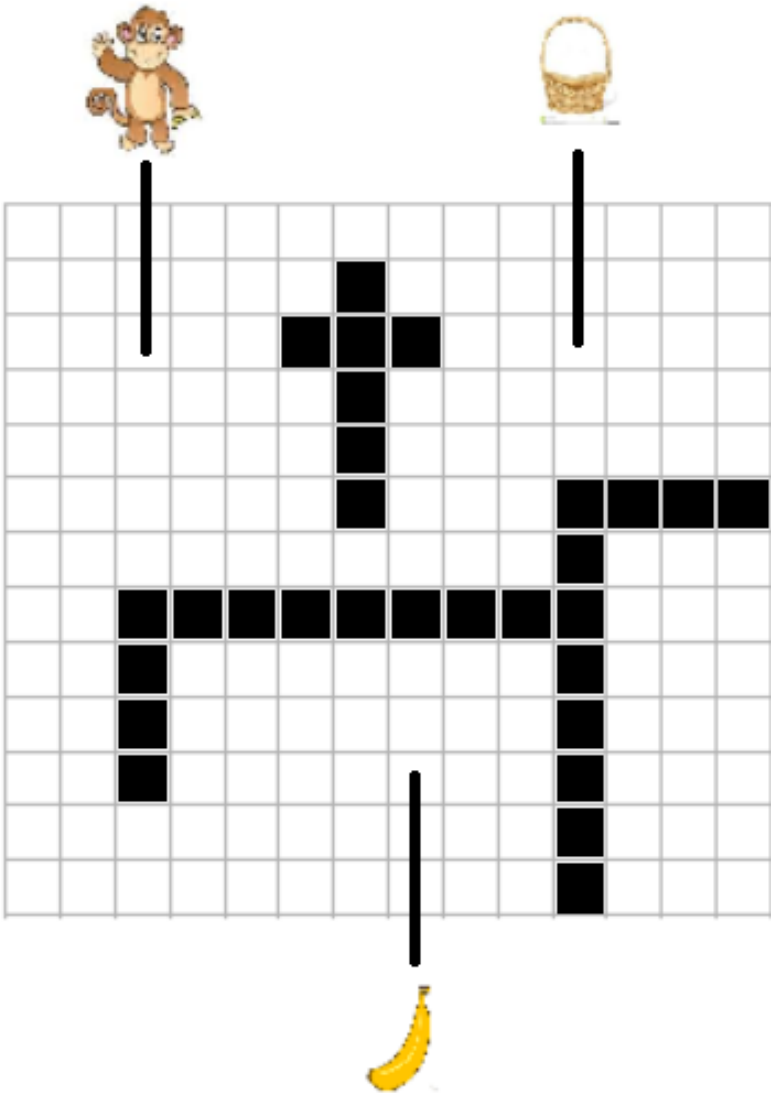
SOLUTION

```
Step 5
Pickup banana
Turn left
Step 5
Turn right
Drop banana
```

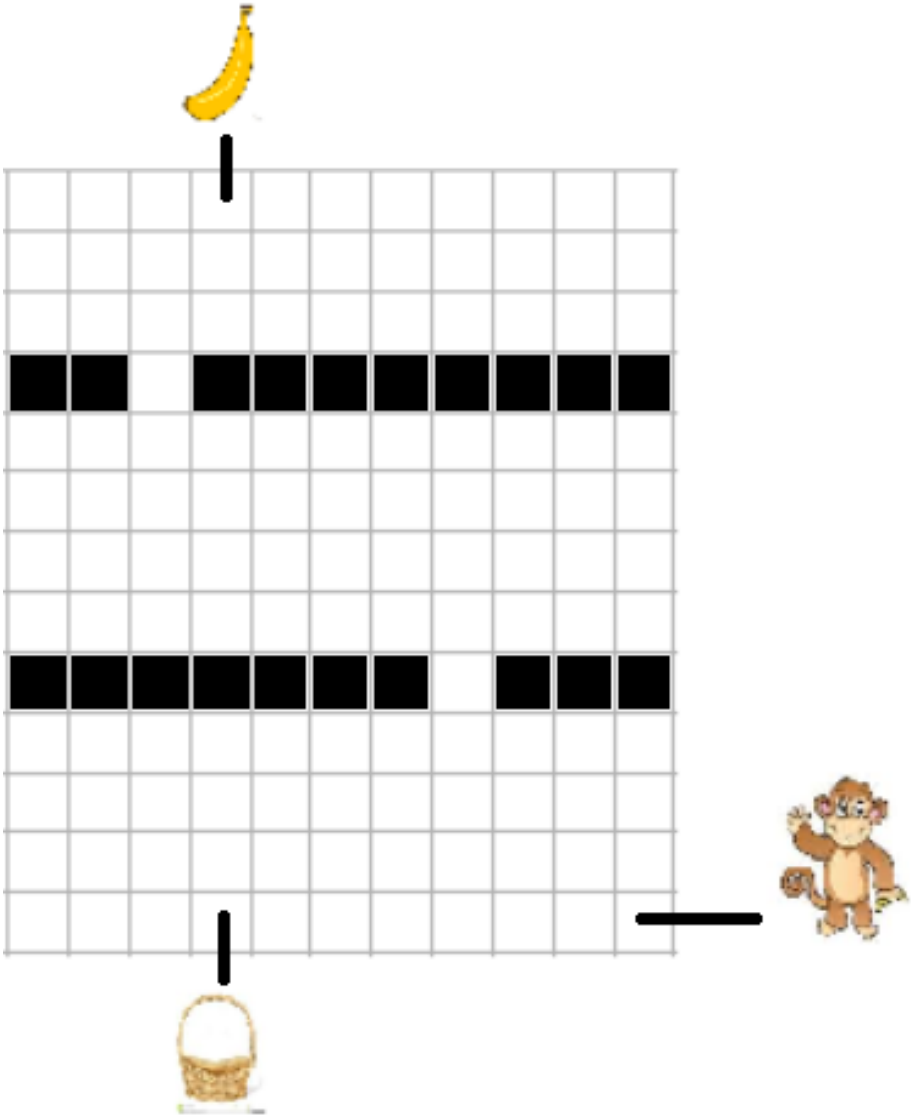
Exercises – Monkey with a Basket

In these exercises, you need to "program your monkey" to put the banana in the basket. Like before, when you finish, talk it over in your group to see how others programmed their monkey.

EXERCISE 1



EXERCISE 2

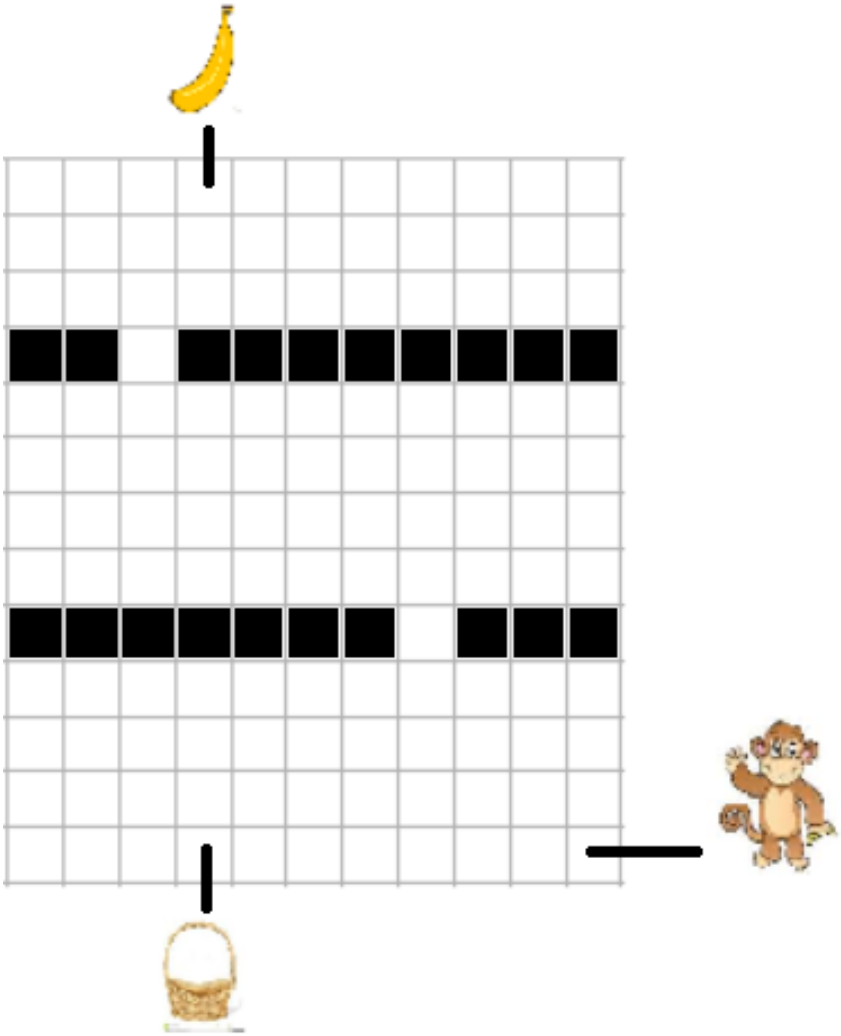


EXERCISE 3

Would your answer change if you knew that the goal was to take the smallest number of steps possible?

The goal here is: *the banana ends up in the basket*.

It doesn't say where the basket needs to be. Do you pick up the banana and carry it to the basket? Do you take the basket over near the banana?



Section 1–4

Dealing with Ambiguity

Dealing with Ambiguity

- **Sometimes, it's harder to generate a list of the tasks you must perform when the process isn't quite as well defined or as visual as working with monkeys!**

- **In these situations, we often write down the steps that we need to perform in English to "get our head around" the problem**
  - When we write down steps for a program, without actual code, this is called pseudocode!

- **Computers follow your exact instructions – but learning to be precise is tricky!**

Trying to Be Exact

- **How do you make scrambled eggs for two people?**

ATTEMPT 1

```
Get four eggs from refrigerator
Crack eggs into skillet
Scramble eggs
```

  - Do you understand the process from our first attempt?
  - What's wrong with the description above?

- **The write up has some pre-conditions to scrambling eggs, but it doesn't really tell us how to scramble eggs**

ATTEMPT 2

```
Get four eggs from refrigerator
Crack eggs into skillet

Repeat
[
   Stir eggs as they cook
   Wait 30 seconds
] until they are the desired consistency

Turn stove off
```

  - Better, but where did the eggs come from?

- **How exact do you have to be?**
  - A lot of that depends on how intelligent the reader is

ATTEMPT 3

```
Open the refrigerator
Get four eggs
Close refrigerator

Crack eggs into skillet

Repeat
[
   Stir eggs as they cook
   Wait 30 seconds
] until they are the desired consistency

Turn stove off
```

- **Of course, if we are programming a computer - it has no preconceived insights and you have to be very exact**

- **You also have to describe possible error conditions**
  - For example, what if we are out of eggs?

ATTEMPT 4

```
Open the refrigerator
If we are out of eggs
[
   Make different breakfast plans
   Close refrigerator
   Exit script
]
Get four eggs
Close refrigerator

Crack eggs into skillet

Repeat
[
   Stir eggs as they cook
   Wait 30 seconds
] until they are the desired consistency

Turn stove off
```

- **Not bad – but are you really being detailed?**
  - Where did the skillet come from?
  - Will your eggs possibly stick to the pan as you cook?

ATTEMPT 5

```
Open the refrigerator
If we are out of eggs
[
   Make different breakfast plans
   Close refrigerator
   Exit script
]
Get four eggs
Close refrigerator

Get skillet from rack above stove
Spray PAM all around skillet for 3 seconds
Put skillet on stove

Crack eggs into skillet

Repeat
[
   Stir eggs as they cook
   Wait 30 seconds
] until they are the desired consistency

Turn stove off
```

- **Can you see your sarcastic friend following the instructions for spraying PAM?**
  - My sarcastic friend would spray the handle and bottom of the skillet!
  - Oops! Replace with:

```
Spray PAM all around the inside of the skillet for 3 seconds
```

Complex Problems

- **When you are working with complex problems, you should think about all aspects of the problem**
  - Where did the PAM come from?
  - Do they know how to crack eggs?
  - What if some of the shell ended up in the pan?
  - What did they do with the shells once they put the egg in the skillet?
  - Who turned on the stove?
  - Stir the eggs with what?
  - What is "desired consistency"?
    - In my house, one of us likes eggs runny and the rest want them cooked into dried hard unappetizing flecks!"
  - Is plating the eggs part of the scope of the problem?
  - Is cleaning up your mess part of the scope of the problem?

- **Other things that might impact how you write include:**
  - The vocabulary of the user (new to the process, experienced)
  - The skills of the user (never cooked, casual cook, chef)
  - The complexity of the process
    - Sometimes, you just won't understand enough about the process to get all of the details right at the beginning
    - Software development is an iterative process!

Exercises – Brush Your Teeth

EXERCISE 1

Write out the process of how to "brush your teeth". This may take 4-10 minutes if you do a good job.

Then, meet with your group to go over the write-ups. Tweak the write-ups as needed.

Your team will present a "brush your teeth" process to the class. Your team can select one of the team member's to advance or can blend them together using steps from each team member.

Module 2

Introducing Java

Section 2–1

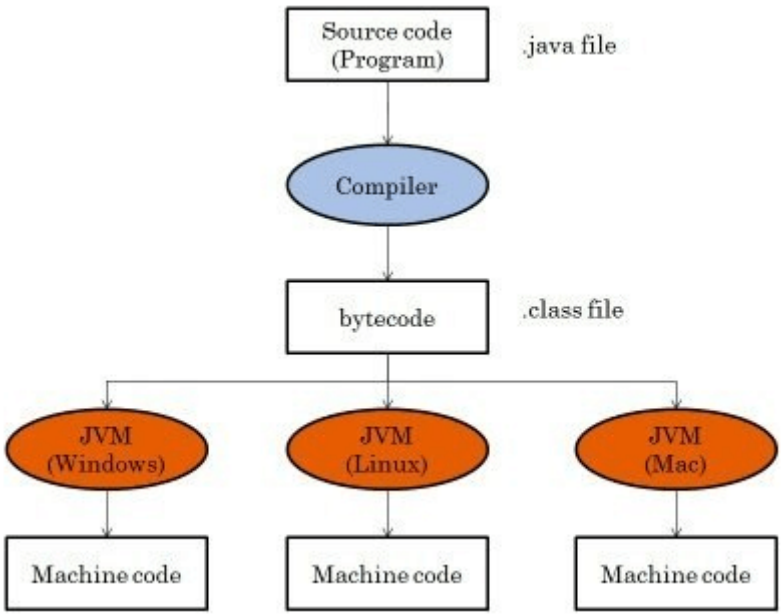
Overview of Java

Java

- **Java is an object-oriented programming language that has been around since the mid 1990s**
  - It was originally developed by Sun Microsystems, and has now been acquired by Oracle
- **One of its strengths is that it is *platform independent***
  - Once translated into Java bytecode, it can run on many different types of computers
  - Often people refer to this as "write once, run anywhere!"
- **Platform independence is what makes Java a popular choice desktops, mobile devices, servers, IoT, and more**

Java Bytecode and the Java Virtual Machine (JVM)

- **When a Java program is compiled, it is not translated into the machine code native to the developer's computer**
  - This would mean it could only run on similar computers
- **Instead, Java programs are translated into something called Java *bytecode***
  - Bytecode is stored in a `.class` file and is eventually executed on a computer by using a JVM
- **The Java Virtual Machine (JVM) is the program responsible for loading and executing a Java application**
  - It executes the Java bytecode instructions
  - There are several JVMs available depending on the type of computer you use



Java Runtime Environment (JRE)

- **You must install the Java Runtime Environment (JRE) in order to *run* Java applications on your computer**

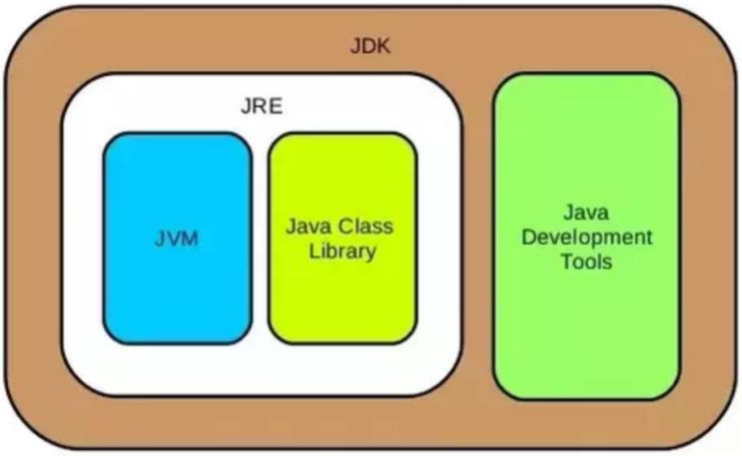
- **The JRE is a software layer that sits on top of a computer's native operating system**

- **It provides the class libraries and other resources (including the JVM) needed to run Java applications**

- **However, the JRE does not contain the resources a developer needs for creating Java applications**

Java Developer Kit (JDK)

- **The Java Developer Kit (JDK) is set of tools for developing Java applications**
  - It includes a CLI, the Java compiler, debuggers, etc.
- **If you want to create Java applications, you must install a JDK**
- **Developers choose JDKs by Java version and by package or edition—**
  - Java Standard Edition (Java SE) <-- We want this one!
  - Java Enterprise Edition (Java EE)
  - Java Mobile Edition (Java ME)
- **The JDK also includes a compatible JRE because so that the Java application can be tested and run on the developer's machine**



Compiling Java

- **To compile a Java application into bytecode, you use `javac`**
- **`javac` is a Java compiler**
  - It confirms the grammar you coded is correct
  - It then translates the code into bytecode and places it in a `.class` file
- **If you have syntax errors, the compiler will not generate the bytecode**

Module 3

Basic Java Syntax

Section 3–1

Basic Java Syntax

Java Syntax

- **Java is one of the C-family languages**
  - Includes C, C++, C#, Java, Kotlin, etc.

- **This means, amongst other things:**
  - it is case sensitive, everywhere and always (!)
  - it uses semicolons (;) as command terminators
  - there are 2 types of comments
    - single line comments
      ```
      //   this line is a comment
      ```
    - and multi-line comments
      ```
      /*
        this line is a comment
        this line is also a comment
      */
      ```
  - the `if`, `switch`, `for`, and `while` statements will look very familiar

- **Source code for Java applications is contained in one or more `.java` files**
  - Unlike most other languages, Java really cares about the name of the file
    - The file must start with an uppercase letter and match the name of the class it contains

Javadoc

- **The Javadoc tool, included in the JDK, generates API documentation from comments found in the source code**

- **Comments should have a specific style**

Syntax

NOTE: a javadoc comment begins with 2 "stars"

```
/**
 * This is a Javadoc comment
 */
```

- **Javadoc looks for these special comments in front of class, method, and variable declarations**

- **These comments are commonly made up of two sections:**
  - Description of what you are commenting
  - Tags marked with an `@` symbol that describe specific metadata

Example: Code Written with Javadoc in Mind

Example

```
/**
 * HelloWorldApp is an example of most people's first
 * Java program
 *
 * @author Dana Wyatt
 *
 */
public class HelloWorldApp {

    public static void main(String[] args) {
        /**
         * The greeting to be displayed in the Console.
         */
        String message = "Howdy Java!";
        display(message);
    }

    /**
     * Dislays any message the console window
     *
     * @param message the message displayed
     */
    public static void display(String message) {
        System.out.println(message);
    }
}
```

Data Types

- **There are two types of data types in Java:**
  - *Primitive* data types include: `boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, and `double`
  - *Non-primitive* data types include arrays, classes, and interfaces

- **Primitive types are pre-defined by Java and represent simple values:**
  - `boolean` – a true/false value in a *single bit*
  - `char` - a single Unicode character (16-bits)
  - numbers such `byte`, `short`, `int`, `long`, `float`, and `double` as that have different memory structures and sizes
    - Note: floats and doubles store their values in an imprecise way - they are not usually used for storing currency values that must be precise

- **Non-primitive types are arrays, class types, etc**
  - They are built by combining primitive types in various ways
  - Java libraries provide many predefined types
  - The `String` data type is actually implemented in a Java class
    - That's why its name starts with a capital S!
  - Programmers also create many classes as they go about developing their application using object-oriented design principles

Variables

- **Java is a statically-typed language and every variable MUST be assigned a data type**

- **Variables in Java are similar to those in other languages**
  - They hold information and allow a programmer to interact with it by the variable name

Example

```
// hold a whole number
int num1;
long num2;

// hold a number with decimal points
float num3;
double num4;

// hold a Boolean
boolean isHappy;
// hold a string
String word;
```

- **You can declare many variables on the same line if they are of the same type**
  - You can choose to initialize some and not others

Example

```
int x = 5, y, z = 50;
```

Data Types: Numbers

- **Java allows programmers to choose from many primitive data types when working with numbers**
  - The difference between them boils down to how many bits are allocated in the computer's memory and how does the computer format those bits

- **For example, a `byte` is an 8-bit number**
  - 7 of the bits are used for the number and 1 bit is used to indicate whether the number is positive or negative
  - This means the range of a `byte` variable is -128 to 127

- **Other whole number types include:**
  - a `short` that is 16 bits and its range is -32,768 to 32,767
  - an `int` that is 32 bits and its range is -2,147,483,648 to 2,147,483,647
  - a `long` that is 64 bits and its range is -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

- **Java's two floating point primitive types are:**
  - a `float` that is 32 bits and holds about 6-7 significant digits
    - Its range is approximately ±3.40282347E+38F
  - a `double` that is 64 bit and holds about 15 significant digits
    - Its range is approximately ±1.79769313486231570E+308

Data Types: Characters

- **Java also contains a `char` type**
  - a `char` is 16 bits and holds a Unicode character - it represents a printable character such as the letter "X"
  - It *can* also be used as an unsigned 16-bit number whose range is 0 to 64,767

Types of Variables

- **In Java, there are three types of variables, depending on *where* and *how* you declare them:**
  - local variables
  - static variables
  - instance variables

- ***Local variables* are declared inside a method**
  - They are created when the method is called and are garbage collected when the method ends
  - Notice that parameters are local variables, too

Example

When `add()` is called, it created the two parameters `num1` and `num2` as well as the variable `sum`. When the `add()` method finishes running, all three variables are destroyed

```
public int add(int num1, int num2) {
   int sum;
   sum = num1 + num2;
   return sum;
}
```

- **The other types of variables will be discussed later in the course**

Read-only Variables

- **Such as a read-only variable, which is declared with the keyword `final`**

- **It must be initialized when you declare the variable**

– Any attempt to change it later will be an error

- **These are sometimes referred to as a constant variable, constant, or final**

- **These variables are non-modifiable after you declare its value, or immutable**

Example

```
final String name = "Dana";

System.out.println("Your name is " + name);

name = name + " Wyatt";  // syntax error!!

System.out.println("Your full name is " + name);
```

Default Values

- **Java provides default values for *class-level* variables based on their data types**
  - Numbers default to 0
  - Booleans default to `false`
  - Objects default to `null`
- **Knowing this explains why code acts as it does when you forget to initialize variables**
- **However, the compiler does NOT assign *local* variables a default**
  - If you try to access a local variable that is *uninitialized*, the compiler will generate a syntax error
- **A best practice says to ***always*** initialize your variables**

Naming Conventions and Consistency

- **Variables use camelCase (lowerCamelCase): first word lowercase, subsequent words capitalized**
  - Examples: `studentName`, `totalPrice`, or `isCurrentlyActive`
  - Be descriptive and consistent: `firstName` not `fName` or `first_name`
- **Classes use PascalCase (UpperCamelCase): each word capitalized with no spaces or separators**
  - Examples: `MathApp.java`, `BankAccount.java`, `StudentRecord.java`
  - File names must match the class name exactly: class `MathApp` must be in `MathApp.java`
- **Packages use all lowercase with no separators between words**
  - Examples: `com.example.myproject`, `org.banking.accounts`
  - Typically follow reverse domain name convention: `com.yourcompany.projectname`
  - Never use spaces, hyphens, or underscores in package names
- **Consistency makes code readable**
- **Mixing conventions makes code harder to read**

Exercise – Variables

EXERCISE 1

Using Notepad declare variables to hold the following data. Consider what information is being stored with each variable, and how you will use the data.

NOTE: this exercise is a mental/paper exercise and **SHOULD NOT** be completed using IntelliJ.

Declare each variable with the correct data types:

- a vehicle identification number in the range 1000000 - 9999999
- a vehicle make /model (i.e. Ford Explorer)
- a vehicle color
- whether the vehicle has a towing package
- an odometer reading
- a price
- a quality rating (A, B, or C)
- a phone number
- a social security number
- a zip code

(you will have a chance to share your answers through zoom)

Section 3–2

Operators and Expressions

String Concatenation

- **Java uses the + operator to concatenate strings**

Example

```
public class BuildAStringApp {

  public static void main(String args[]){
     String word1 = "Hello";
     String word2 = "World";
     String greeting;

     greeting = word1 + " " + word2 + "!";
     System.out.println(greeting);
  }
}
```

OUTPUT

```
Hello World!
```

- **When you Google, you will also find that there is a `concat()` method**

Mathematical Operators and Expressions

- **Java supports the standard arithmetic operators you find in most languages (+ – * /) and well as the remainder (or *modulo*) operator (%)**

- **Because Java is strongly typed, the results of arithmetic operations are sometimes surprising**
  - Note: one integer divided by another is an integer!

Example

```
public class BasicIntegerMathApp {
   public static void main(String[] args){
      int a = 10;
      int b = 3;
      int result;
      result = a + b;
      System.out.println(result);  // displays 13
      result = a - b;
      System.out.println(result);  // displays 7
      result = a * b;
      System.out.println(result);  // displays 30
      result = a / b;
      System.out.println(result);  // displays 3
      result = a % b;
      System.out.println(result);  // displays 1
  }
}
```

- **Floating point math always returns a floating point result**

Example

```
public class BasicFloatingPointMathApp {

   public static void main(String[] args){
      float a = 10;
      float b = 3;
      float result;
      result = a + b;
      System.out.println(result);  // displays 13.0
      result = a - b;
      System.out.println(result);  // displays 7.0
      result = a * b;
      System.out.println(result);  // displays 30.0
      result = a / b;
      System.out.println(result);  // displays 3.3333333
      result = a % b;
      System.out.println(result);  // displays 1.0
      result = b - (a % b);
      System.out.println(result);  // displays 2.0
  }
}
```

- **Parentheses can be placed around expressions to control the order in which they are evaluated**
  - Expressions can be a combination of integer and floating point values, but if the result is a float point value, it must be assigned to a floating point variable

Pre/Post - Increment and Decrement

- **Like other C-based languages, Java embraces the pre/post-increment and decrement operator**

Example

```
int x = 5;
int y;

x++;   // adds 1 to x   (now it is 6)
++x;   // adds 1 to x   (now it is 7)

y = ++x;    // adds 1 to x and then assigns x to y
            // x is now 8 and y is 8

x = 5;

y = x++;    // assigns x to y and then adds 1 to x
            // x is now 6 and y is 5
```

Literals Are Typed

- **In Java, literal values (hard coded values) have a data type**

Example

```
101   <- an int
101L  <- a long (can use a little l, but it looks like a 1)
7.25  <- a double
7.25f <- a float
"Hi"  <- a String
'a'   <- a char
true  <- a boolean
```

- **Starting in Java SE 7, you can place underscore characters (_) between digits in a numerical literal**
  - This sometimes makes the number more readable

Example

```
long ssn = 111_22_3333L;

long creditCard = 5200_7500_6500_0001L;

double loanBalance = 1_225_570.00;
```

- **You can't place underscores:**
  - At the beginning or end of a number
  - Adjacent to a decimal point in a floating point literal
  - Prior to an `F` or `L` suffix

- **When using literals, you must decide whether you want to use 32-bit floating point values…**

Example

```
public class PriceApp {

   public static void main(String args[]){
      int count = 11;
      float unitPrice = 7.12f; //this is an error without the f
      float taxRate = 0.825f;
      float totalCost;

      totalCost = (count * unitPrice) * (1 + taxRate);
      System.out.println(totalCost);
   }
}
```

- **… or 64 bit ones**

Example

```
public class PriceApp {
   public static void main(String args[]){
      int count = 11;
      double unitPrice = 7.12;
      double taxRate = 0.825;
      double totalCost;
      totalCost = (count * unitPrice) * (1 + taxRate);
      System.out.println(totalCost);
   }
}
```

- **Static typing will take some getting used to!**

Widening Issues

- **When you take a variable or expression and assign it to a variable of a wider data type, there are no errors and it is considered a "safe" process**

- **What does wider mean?**
  - It means no loss of precision

`byte -> short -> char -> int -> long -> float -> double`

- **For example, assigning a `float` variable to a `double`!**
  - … or an `int` variable to a `long`!

Example

```
int myInt = 9;
long myLong;

float myFloat = 3.8f;
double myDouble;

// an int fits in a long
myLong = myInt;

// a float fits in a double
myDouble = myFloat;

// you can even put an int or long into a float or double
myFloat = myLong;
myDouble = myLong;
```

- **Widening isn't about a specific *value* for a variable and whether it will fit in the new variable**
  - It is about whether any value in the source *data type* will fit in the data type of the new variable

Narrowing Issues

- **When you take a variable or expression and assign it to a narrower variable, the compiler will generate an error**
  - If you want the assignment to happen, you must explicitly tell the compiler

- **What does narrower mean?**
  - It means there is a "chance" that a value in the source variable might now fit in the narrower variable

`double -> float -> long -> int -> char -> short -> byte`

- **For example, assigning a `double` variable to a `float`!**
  - … or a `float` variable to a `int`!

Example

```
int myInt;
long myLong = 9;

float myFloat;
double myDouble = 123.4567890123;

// an int won't necessarily fit in a long
myInt = myLong;

// a double won't necessarily fit in a float
myFloat = myDouble;

// a float probably won't necessarily fit in an int
myLong = myFloat;
```

- **SYNTAX ERRORS**

```
Main.java:12: error: incompatible types: possible lossy conversion from long to int
myInt = myLong;
        ^
Main.java:15: error: incompatible types: possible lossy conversion from double to float
myFloat = myDouble;
          ^
Main.java:18: error: incompatible types: possible lossy conversion from float to long
myLong = myFloat;
         ^
3 errors
```

  - Syntax errors can be caused from a number of different reasons
  - This output shows us that when we are trying to convert from one data type to another, we are having incompatibility issues
  - Don't be afraid to read errors, they are our friends!
    - Start by taking in what you know, then try to fill in the gaps for what you don't

Type Casting

- **Sometimes you know that a narrowing assignment is okay and can force the compiler to make the assignment using type casting**

- **To cast an expression to a narrow type, put the target type in parenthesis in front of the expression**
  - It will suppress the compiler error

Example

```
long myLong = 9;
int myInt;

// you know the range of values in the long variable
// will always fit in the int
myInt = (int) myLong;
```

- **This is often needed because you are calling a function whose return type you can't control**
  - For example, you are calling `Math.pow()` that returns a double but you need less precision

    ∗ See more details on the next page

Example

```
float num = 2.2;

// if you don't cast the result, you have to store
// the value in a double variable
float result = (float) Math.pow(num, 2);
```

Java's `Math` Class

- **Java has a `Math` class with many helpful predefined methods**
  - See: w3schools.com/java/java_ref_math
- **The methods are static methods**
  - This means you invoke them using the name of the class

Example

```
double degrees = 90.0;

double answer = Math.sin(degrees);
```

- **Most of the methods have a one or more double parameters and return a double**
  - But read the documentation because this isn't entirely true

Example

```
double value = 1234.567;

int wholeNumber = Math.round(value);
```

Example: Working with the `Math` Class

Example

```
public class MathApp {
   public static void main(String[] args) {
      int natKidCount = 2, brittKidCount = 4;

      int mostKids = Math.max(natKidCount, brittKidCount);

      System.out.println("The biggest family has " + mostKids + " kids");
   }
}
```

Assignment Operators

- **Java supports a set of assignment operators that are combined with arithmetic operators to make certain operations easier to express**

- **They include: += -= *= /= %=**

Example

```
// Original technique

int answer = 0;
answer = answer + 10; // 10
answer = answer - 5;  // 5
answer = answer * 10; // 50
answer = answer / 2;  // 25
answer = answer % 10; // 5  (int remainder of 25 / 10)
```

Example

```
// Shortcut technique

int answer = 0;
answer += 10; // 10
answer -= 5;  // 5
answer *= 10; // 50
answer /= 2;  // 25
answer %= 10; // 5  (int remainder of 25 / 10)
```

Exercises – Operators

In this exercise you will create a simple project that uses java to perform various mathematical calculations. Complete your work in the `pluralsight/workbook-1` folder.

EXERCISE 2

Using IntelliJ create a new Java project named string-application.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always share your IntelliJ project to GitHub

Add a new package named `com.pluralsight` with a file named `VariableApp.java` structured as follows:

```
public class VariableApp {
   public static void main(String[] args) {

      // Question 1:
      // declare variables here
      // then code solution
      // then use System.out.println() to display results
      // ex: System.out.println("The answer is " + answer);

      // REPEAT FOR NEXT EXERCISE

      }
}
```

Step 1

Write code to print each to the following questions.

Questions:

1. Declare a variable to store your favorite color
2. Declare a variable to store the year you started this class
3. Declare a variable to store your middle initial
4. Declare a variable to store whether you have pets (yes/no)
5. Declare a variable with a nice message or saying

Step 2

1. Declare AND initialize variables for:
2. The number of days in a week
3. The price of coffee ($4.99)
4. Your favorite letter
5. Whether it's raining (true/false)
6. **Print all values in complete sentences**

Step 3

Create a new file in `com.pluralsight` named `Reciept.java` structured as follows:

```
public class Reciept {
   public static void main(String[] args) {

        System.out.println();

     }
}
```

1. Create variables for an item name (String) and item price (double)
2. Create a quantity variable (int)
3. Build and print: EX. "You bought 3 apples for $7.10"

EXERCISE 3

Using IntelliJ create a new Java project named math-application.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always create the Github repository before creating a new Java project.

Add a new package named `com.pluralsight` with a file named `MathApp.java` structured as follows:

```
public class MathApp {
   public static void main(String[] args) {

      // Question 1:
      // declare variables here
      // then code solution
      // then use System.out.println() to display results
      // ex: System.out.println("The answer is " + answer);

      // REPEAT FOR NEXT EXERCISE

      }
}
```

Step 1

Write code to find answers to the questions on the following page.

NOTES: Coding is not just about solving equations, your code should be legible and meaningful

Use comments and line spacing to visually separate each question.

Take your time to think of and use meaningful variable names (don't just use num1)

Print **meaningful** messages to the screen so that the reader has all of the context information for what you are.

QUESTIONS:

1. Create two variables to represent the salary for Bob and Gary, name them `bobSalary` and `garySalary`. Create a new variable named `highestSalary`. Determine whose salary is greater using `Math.max()` and store the answer in `highestSalary`. Set the initial salary variables to any value you want. Print the answer (i.e "The highest salary is …")

2. Find and display the smallest of two variables named `carPrice` and `truckPrice`. Set the variables to any value you want.

3. Find and display the area of a circle whose radius is 7.25

4. Find and display the square root a variable after it is set to 5.0

5. Find and display the distance between the points (5, 10) and (85, 50)

6. Find and display the absolute (positive) value of a variable after it is set to -3.8

7. Find and display a random number between 0 and 1

8. Calculate how many minutes are in 24 days, use a variable for each value you calculate with. **BONUS:** How many milliseconds?

Step 2

1. Ensure all your changes are committed and pushed to GitHub

2. Send your repository URL to your Instructor

Section 3–3

Writing to the Screen and Reading from the Keyboard

Writing Text to the Screen

- **You can use both the `System.out.print()` and `System.out.println()` methods to display a line of text**
  - The difference is that the `println()` method places a CR/LF in the output stream after it writes

Example

```
class Program
{
   public static void main(String args[])
   {
      System.out.print("Hello ");
      System.out.print("World!");
   }
}
```

OUTPUT

```
Hello World!
```

Example

```
class Program
{
   public static void main(String args[])
   {
      System.out.println("Hello World!");
   }
}
```

OUTPUT

```
Hello World!
```

- **The two examples above aren't *exactly* the same…**
  - QUESTION: What would happen if you added a `println()` statement to each?

Formatting Output

- **There are several ways to format numbers in Java**
  - java2blog.com/format-double-to-2-decimal-places-java

- **One of the easiest is to use the `String` class' `format()` method**
  - It accepts a format string (more on this on the next page)

Example

```
float subtotal = 22.87;
float tax = subtotal * 0.0825f;
float totalDue = subtotal + tax;

System.out.println(
    "Total due is: " + String.format("%.2f",  totalDue));
```

- **Another way is to use the `printf()` method instead of `print()` or `println()`**
  - It accepts a format string also

Example

```
float subtotal = 22.87;
float tax = subtotal * 0.0825f;
float totalDue = subtotal + tax;

System.out.printf("Total due is: %.2f",  totalDue);
```

Java Format Specifiers

- **Format specifiers begin with a percent character (`%`)**
- **They end with a type character that specifies type of data (`int`, `float`, etc) being formatted**
- **In between, you can specify width and precision**

Syntax

```
% [flags] [width] [.precision] type-character
```

- **Type characters include:**
  - `d` decimal integer (base 10 number)
  - `x` or `X` hexadecimal integer (base 16)
  - `f` floating point number
  - `s` string
  - `c` character

Example

```
int id = 10135;
String name = "Brandon Plyers";
float pay = 5239.77f;

System.out.printf("%s (id: %d) $%.2f", name, id, pay);
```

Reading Input with Scanner

- **You can use the `Scanner` class to read input**
  - A whole line at a time, or
  - One formatted value at a time

- **The `Scanner` class is defined in `java.util.Scanner` and you will have to import it at the top of the class file**
  - ∗ You can import just the `Scanner` class or all classes in the same package

Example

```
java.util.Scanner;      // import the Scanner class
import java.util.*;     // import all classes in the package
```

- **If you pass `System.in` as a constructor argument, the Scanner will read from the keyboard**

Example

```
Scanner scanner = new Scanner(System.in);
```

Read a Whole Line of Text

- **Use the `nextLine()` method to read a line of input**
  - Scanner reads the input up to the next line separator (LF or CRLF) and returns it as a String
  - The line separator character is discarded
  - If the scanner is closed, it throws an **`IllegalStateException`**

Example

```
import java.util.*;

class Program {
   public static void main(String[] args) {
      Scanner myScanner = new Scanner(System.in);
      System.out.print("Enter your name: ");
      String name = myScanner.nextLine();
      System.out.println("Howdy " + name);
   }
}
```

Read Individual Values

- **To read individual data values (numbers, words, booleans, etc.) you can use other `Scanner` methods; for example:**
  - `nextInt()`
  - `nextFloat()`
  - `nextDouble()`
  - `nextBoolean()`

- **By default, the scanner breaks the input into *tokens*, separated by *whitespace* (space, tab, line separators, etc.)**

- **Scanner reads the next token and returns its value, *if possible*, as the desired type**
  - Scanner may throw an `InputMismatchException` if the input is not compatible with the requested type
  - *Initial* whitespace is discarded, but *following* whitespace is left in the stream (more on this in a minute)

Example

```
System.out.println("How old are you? ");
int age = scanner.nextInt();

System.out.println("What is your salary? ");
double salary = scanner.nextDouble();
```

Example: A Calculator

Example

```
import java.util.*;

class CalculatorApp
{
   public static void main(String[] args)
   {
      Scanner myScanner = new Scanner(System.in);

      // get two numbers, add them together, and display the sum

      System.out.print("Enter first number: ");
      int num1 = myScanner.nextInt();

      System.out.print("Enter second number: ");
      int num2 = myScanner.nextInt();

      int sum = num1 + num2;

      System.out.println("The sum is " + sum);
   }
}
```

Mixed Input Types

Example

File: UserInputApp.java

```
import java.util.*;
public class UserInputApp {
    public static void main(String[] args) {
        // create a scanner
        Scanner scanner = new Scanner(System.in);
        // read name
        System.out.print("What is your name? ");
        String name = scanner.nextLine();
        // read age
        System.out.print("How old are you? ");
        int age = scanner.nextInt();
        // display a greeting
        String greeting;
        if (age <= 21) {
            greeting = "Hi ";
        } else {
            greeting = "Hello ";
        }
        System.out.println(greeting + " " + name + "!");
    }
}
```

TRACE OF TERMINAL WINDOW SESSION

```
What is your name? Dana
How old are you? 63
Hello Dana!
```

- **It seems to work fine, but we must be careful...**

Buffered Input

- **Most operating systems require you to hit the ENTER key to send input on through to the program**
  - This actually inserts a line separator (Usually CRLF) in the input buffer, which must be handled by the scanner

Example

```
Scanner scanner = new Scanner(System.in);

System.out.print("How old are you? ");
int age = scanner.nextInt();
```

TRANSCRIPT OF RUNNING CODE:

```
How old are you? 63↵    where the return arrow
                           is the ENTER key (CRLF)
```

- **In this example, `nextInt()` reads 63 correctly, and leaves the following CRLF in the input buffer**

- **Does this cause a problem?**
  - It depends...

Line Separators and Numeric Input

- **If another numeric input method is called (`nextInt()`, `nextFloat()`, etc), it discards the leftover CRLF as whitespace before reading the next token**
  - It's looking for numbers -- so an extra CRLF doesn't affect numeric input

Example

```
Scanner scanner = new Scanner(System.in);

System.out.print("Enter number 1: ");
int num1 = scanner.nextInt();

System.out.print("Enter number 2: ");
int num1 = scanner.nextInt();
```

TRANSCRIPT OF RUNNING CODE:

```
Enter number 1: 13↵
Enter number 2: 145↵    This ignores the CRLF left in the
                           buffer after reading 13 and
                           successfully reads 145

// HOWEVER, now there is a different CRLF
// left in the buffer after 145
```

Line Separators and nextLine()

- **If the next read is performed with `nextLine()`, it stops at the first CRLF it finds!**

Example

```
Scanner scanner = new Scanner(System.in);

System.out.print("Enter number 1: ");
int num1 = scanner.nextInt();

System.out.print("Enter number 2: ");
int num2 = scanner.nextInt();

System.out.println("What do you want to do? ");
System.out.print("   Enter 'add' or 'subtract': ");
String action = scanner.nextLine();

System.out.println("Preparing to do math... ");
```

TRANSCRIPT OF RUNNING CODE:

```
Enter number 1: 13↵
Enter number 2: 145↵
What do you want to do?
   Enter 'add' or 'subtract': Preparing to do math...

   // The system didn't wait for the user to enter
   // data because the CRLF in the buffer after 145
   // marked the end of the input when nextLine()
   // went to read its data
```

Dealing With Line Separators

- **So how do you fix this?**
  - Carefully and on a case-by-case basis
- **If you read a mix of numbers and text, whose job is it to get rid of the extra CRLF???**
  - The code reading the number (that left CRLF in the buffer)?
  - The code reading text (that doesn't want a CRLF before its read)?
- **But --**
  - If the code reading an int gets rid of the CRLF, it would be wasted energy if the next piece of code also reads an int
  - If the code reading text tries to get rid of the CRLF and its not there, then that could be a problem
- **There's no great answer! But someone has to decide. So, here is my answer for the code above…**
  - If will forcibly consume the CRLF that is causing the problem!

Consuming a Line Separator

Example

```
Scanner scanner = new Scanner(System.in);

System.out.print("Enter number 1: ");
int num1 = scanner.nextInt();

System.out.print("Enter number 2: ");
int num2 = scanner.nextInt();
scanner.nextLine();   // "eat" the leftover CRLF

System.out.println("What do you want to do? ");
System.out.print("   Enter 'add' or 'subtract': ");
String action = scanner.nextLine();

System.out.println("Preparing to do math... ");
```

TRANSCRIPT OF RUNNING CODE:

```
Enter number 1: 13↵
Enter number 2: 145↵
What do you want to do?
   Enter 'add' or 'subtract': add↵
Preparing to do math...
```

Exercises – Basic Calculator

In these exercise you will create basic console applications which prompt the user for information and then perform calculations based on the user input. Remember to do your work in the `workbook-1` directory.

Remember to be focus on the format and readability of your code.

a. Use meaningful variable names

b. Use whitespace and lines to organize your code and thoughts so that others can easily understand your code

c. Don't try to do multiple things on a single line of code

d. When in doubt opt for code readability

EXERCISE 3

Create a Java application named basic-calculator that reads in two floating point numbers and then asks the user whether they want to add, subtract, multiply or divide the two numbers.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always share your IntelliJ project to GitHub

For this exercise, only code the questions. We won't calculate add, subtract, multiply or divide. For this exercise, **always multiply**.

Step 1

Perform the requested operation and display the results.

```
Enter the first number: 5
Enter the second number: 12

Possible calculations:
  (A)dd
  (S)ubtract
  (M)ultiply
  (D)ivide
Please select an option: M

5 * 12 = 60
```

Step 2

1. Ensure all your changes are committed and pushed to GitHub

EXERCISE 4

Step 1

Create a Java application named payroll-calculator that prompts the user to enter:

- their name
- their hours worked (a floating point number)
- their pay rate (a floating point number) Calculate their gross pay.

Display the employee's name and their gross pay.

Step 2

1. Ensure all your changes are committed and pushed to GitHub
2. Send your repository URL to your Instructor

(Optional)

You learned about `if` statements in your pre-work, so although we have not discussed them in class, you should be able to figure out how to pay 1.5x overtime after 40 hours. You can use W3Schools guide as a reference
https://www.w3schools.com/java/java_conditions.asp

Learning Effectively

- **What makes learning programming different?**
  - Programming is a skill that builds on itself, each concept connects to the next
  - It requires both theoretical understanding AND hands-on practice
  - You'll make lots of mistakes, and that's part of how you learn
- **Developing a growth mindset for coding**
  - Expect to feel confused sometimes, this is normal and hopefully temporary
  - Every programmer started exactly where you are
  - The goal isn't to memorize syntax, but to understand problem solving patterns
- **How to approach problems in this course**
  - Read requirements carefully before jumping into code
  - Start with small, simple examples before tackling complex problems
  - Test your code frequently as you build it
  - When stuck, first try to identify exactly what you don't understand

- **Building good learning habits**
  - Practice coding regularly (even 15-20 minutes daily beats one long session)
  - Take notes on concepts, not just syntax
  - Explain your code out loud
  - If you can't explain it, you don't fully understand it
  - Collaborate with classmates, but make sure you understand every line you write

AI and Learning Effectively

- **What is AI?**
  - They are computer systems and models that perform tasks requiring human intelligence
  - Large language models predict and generate human-like text responses
  - They have their own limitations: potential inaccuracies, knowledge gaps, lacks true understanding
- **AI as a learning tool**
  - It *enhances* rather than replaces critical thinking
  - It is best when you have foundational knowledge to evaluate suggestions
  - Some Tools include Claude, ChatGPT, Gemini, and lots of others
- **How using AI enables us to learn more effectively**
  - When you know how to break down problems yourself, you can ask AI better questions
  - Understanding your learning process helps you identify where AI can truly help vs. where it might shortcut important practice
  - Good debugging habits mean you can spot when AI suggestions don't quite fit your specific problem
  - The ability to explain concepts out loud translates directly to asking AI for explanations rather than solutions
  - Knowing when you're truly stuck (vs. just being impatient) helps you use AI as a learning tool rather than a crutch

AI Ethics

- **Ethics and responsibility**
  - AI can exhibit bias and produce incorrect but plausible information
  - It's crucial to have academic integrity through transparent use of these tools
  - Always fact check
  - **Never submit code you don't understand**
    - If you find code somewhere, online or through AI, you are responsible for understanding every line before submitting it
    - **Being able to explain your code is as important as coding**
    - Copy without comprehension creates bigger problems down the road, bugs you can't fix, vulnerabilities you can't see
  - Be mindful of what you share with AI tools, avoid pasting sensitive, private, or proprietary data into public models
  - Attribution matters, be transparent about when and how you used AI assistance

AI Ethics

- **Course applications**
  - We will use AI for research and brainstorming support
  - It will allow us to increase our problem solving skills
  - We can use AI to continue to learn better questioning skills
    - The quality of your prompt directly impacts the quality of the output
    - Learning to ask precise, and well-structured questions is a skill that will transfer beyond AI
  - Appropriate uses include rubber duck debugging, explaining error messages, generating test cases, and exploring new concepts
  - AI is not always right, it can reference outdated libraries, incorrect syntax, and insecure patterns
  - **We will use AI as a thinking partner, not a replacement for understanding the material**

Example: Using AI Effectively

Hey! Can you help me decide on whether to use a char or String? I'm not exactly sure about the difference. I'm learning Java

Absolutely — I'd be happy to help! You're going to encounter both `char` and `String` a lot in Java, so it's great you're digging into this early.

✅ Use `char` when:

- You only need **one character**.
- You're working with low-level character operations (e.g., comparing individual letters, checking if a character is a digit).
- Example:

```java
char letter = 'A';
if (letter >= 'A' && letter <= 'Z') {
    System.out.println("It's an uppercase letter.");
}
```

✅ Use `String` when:

- You need **text** (words, sentences, names, inputs).
- You're performing string operations (concatenation, search, replace, etc.).
- Example:

```java
String name = "Alice";
System.out.println("Hello, " + name + "!");
```

🚫 Common mistakes:

- Writing `'hello'` → **Error!** That's not a `char` (too many characters).
- Writing `"a"` when you need a `char` → `"a"` is a `String`, not a `char`.

🧠 Quick way to remember:

- `char` = one letter (`'x'`)
- `String` = a word or sentence (`"example"`)

Let me know if you want examples where you'd switch between the two — or if you're doing a specific project and not sure which fits better!

Module 4

Conditionals and Methods

Section 4–1

Conditionals

Conditionals

- **Conditional logic lets us do different things depending on the value of an expression**

- **Conditionals include:**
  - `if` and `if/else` statements
  - `switch` statements
  - conditional operator (sometimes called the ternary operator)

`if` Statement

- **The if statement executes some code if a condition expression evaluates as `true`**
  - The condition is surrounded by parentheses
  - Curly braces are only *required* around the code if there is more than one statement, but best practice is to always have them

Syntax

```
if (condition) {
   // statement(s)
}
```

- **The condition is often expressed by comparing values:**
- **== (equals) != (not equals)**
- **< <= > >=**
  - The equality and inequality operators can be used with both primitive types and objects
  - The other comparison operators only work with primitive types that can be represented in numbers

Example

```
int age = 17;
double price = 25.00;

if (age < 18) {
   price = price * .9;
}
System.out.println("Price of admission: " + price);
```

if / else

- **The `else` statement can be used to write alternative code that executes when the condition is false**
  - If the else condition has more than one statement, you must surround them with curly braces
  - `else` must always follow an `if`

Example

```
String name = "Ezra";
int age = 17;

double price;

if (age < 18) {
   price = 22.50;
} else {
   price = 25.00;
}

System.out.println("Price of admission: " + price);
```

- **You can string together `if / else if / else` statements for more complex logic**

Example

```
String name = "Ezra";
int age = 17;

double price;

if (age < 18) {
   price = 18.00;
} else if (age < 65) {
   price = 25.00;
} else {
   price = 18.00;
}

System.out.println("Price of admission: " + price);
```

- **You can build complex conditions by using the logical operators: `&&` (and) `||` (or) `!` (not)**

Example

```
String name = "Ezra";
int age = 17;

double price;

if (age < 18 || age >= 65) {
   price = 18.00;
} else {
   price = 25.00;
}
```

Comparing Strings

- **The == operator tests whether both operands are the same (identical) object**
  - This doesn't work reliably for Strings, since the string you read as input is a different object from the one that holds a literal value

- **The String class's `equals()` method tests whether the strings contain the same characters in the same order**

Example

```
String homeState = "Texas";
String contactPhone;

if (homeState.equals("Texas") || homeState.equals("Kansas")) {
   contactPhone = "800-555-5555";
} else {
   contactPhone = "855-555-5555";
}
```

Example: Better Calculator?

Example

```
import java.util.*;

public class InputApp {

   public static void main(String[] args) {

     Scanner scanner = new Scanner(System.in);
     System.out.print("What do you want to do (add, subtract) ? ");

     String command = scanner.nextLine();
     if (command.equals("add")) {
         doAdd(scanner);
     } else if (command.equals("subtract")) {
         doSubtract(scanner);
     } else {
         System.out.printf("%s -- Invalid command!", command);
     }
  }

   public static void doAdd(Scanner scanner) {
     System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double sum = num1 + num2;
     System.out.printf("%f + %f = %f", num1, num2, sum);
   }

   public static void doSubtract(Scanner scanner) {
     System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double difference = num1 - num2;
     System.out.printf("%f + %f = %f", num1, num2, difference);
   }
}
```

Exercises – Sandwich Shop

In these exercises you will create a Sandwich Shop application. You will allow users to order a simple sandwich and you will calculate the price. The customer may receive a discount based on their age.

Remember to create your projects in the `pluralsight/workbook-1` folder.

EXERCISE 1

Create a Java application named sandwich-shop. This will be a point of sales application to calculate the cost of a sandwich.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always share your IntelliJ project to GitHub

Step 1

Prompt the user for the size of the sandwich (1 or 2):

a. 1: Regular: base price $5.45

b. 2: Large: base price $8.95

Prompt the user for their age:

a. Student (17 years old or younger) – receive a 10% discount

b. Seniors (65 years old or older) – receive a 20% discount

Display the cost of the sandwich to the screen

Step 2

1. Ensure all your changes are committed and pushed to GitHub
2. Send your repository URL to your Instructor

EXERCISE 2

Modify the sandwich-shop from the last exercise to allow customers to order a "loaded" sandwich (double everything).

Step 1

Prompt the user for the size of the sandwich (1 or 2):

a. 1: Regular: base price $5.45

b. 2: Large: base price $8.95

Prompt the user if they would like the sandwich "loaded" (yes/no). If so there is an additional cost for a loaded sandwich

a. Regular: $1.00

b. Large: $1.75

Prompt the user for their age:

a. Student (17 years old or younger) – receive a 10% discount

b. Seniors (65 years old or older) – receive a 20% discount

Display the cost of the sandwich to the screen

Step 2

1. Ensure all your changes are committed and pushed to GitHub

2. Send your repository URL to your Instructor

Section 4–2

Static Methods

Static Methods

- **You can add static methods to your application class besides the required `main()` method**

- **Static methods do not require us to instantiate the class**
  - Since we are first learning Java, they allow us to break our application logic up into more manageable "chunks"
  - Eventually, we will transition to fully object-oriented coding

- **Since the class may not be instantiated, a static method is not allowed to access instance variables**
  - It can only access local variables or static variables

- **For now, ignore the fact that these are static methods and you can refer to these as methods**
    - You'll learn more about this later in the course

Declaring a static Method

- **Methods perform an action**
  - A method can create an effect

```
public static void add() {
   System.out.println(2 + 2);
}

add(); //prints 4
```

  - A method can optionally return a response

```
public static int add(int x, int y) {
    return x + y; //returns y
}

System.out.println(add(2, 2)); //prints 4
```

- **A method is used to break up your program into logical operations**

- **Methods should do one thing**
  - Just like an appliance, we can reuse methods and have the same expected results
  - Methods also enable the use of the D.R.Y. method
  - **Don't Repeat Yourself**

Method Responses

- **Methods, like variables must have a data type set**
  - These data types determine the type of data the method will return
    - Just like a variable will have a value when you use it, such as printing or adding with that variable

```
public static int add(int x, int y) {
    return x + y; //returns y
}

int numbersAdded = 2 + 2; //4

int addedResult = add(2 + 2); //4
```

  - If a method doesn't return anything, it uses the word `void` as the return type

Example

```
public class MainApp {

   public static void main(String[] args) {
      foo(); //prints Foo
      moo(); //prints Moo
   }

   public static void foo() {
      System.out.println("Foo");
   }

   public static void moo() {
      System.out.println("Moo");
   }
}
```

OUTPUT

```
Foo
Moo
```

Passing Data to Methods

- **You can pass data to a method in Java**
  - The data is referred to as **arguments**
  - Within the methods parenthesis, we declare the parameters that will contain the arguments value when the method is invoked
  - Arguments and parameters don't have to have the same name, but they should be of the same type

Example

MainApp.java

```
public class MainApp {

   public static void main(String[] args) {
      int a = 4, b = 9, c = 10, d = 3;

      addAndDisplay(a, b);
      addAndDisplay(b, c);
      addAndDisplay(a, d);
   }

   public static void addAndDisplay(int num1, int num2) {
      int sum = num1 + num2;
      System.out.printf("%d + %d = %d", num1, num2, sum);
   }
}
```

OUTPUT

```
4 + 9 = 13
9 + 10 = 19
4 + 3 = 7
```

Example

MainApp.java

```
public class MainApp {

   public static void main(String[] args) {

      int difference = subtract(10, 5);
      int sum = add(2, 2);

      System.out.println(difference);
      System.out.println(sum);
   }

   public static int subtract(int x, int y) {
      return x - y;
   }

   public static int add(int x, int y) {
      return x + y;
   }

}
```

OUTPUT

```
5
4
```

Returning Data from Methods

- **Java requires you to specify the type of data that will be returned from a method**
  - Instead of `void`, you would specify the data type of the value being returned

Example

MainApp.java

```
public class MainApp {

   public static void main(String[] args) {
      int a = 4, b = 9, c = 10, d = 3;
      int sum;

      sum = add(a, b);
      display(a, b, sum);

      sum = add(b, c);
      display(b, c, sum);

      sum = add(a, d);
      display(a, d, sum);
   }

   public static int add(int num1, int num2) {
      int sum = num1 + num2;
      return sum;
   }

   public static void display(int num1, int num2, int sum) {
      System.out.printf("%d + %d = %d", num1, num2, sum);
   }
}
```

OUTPUT

```
4 + 9 = 13
9 + 10 = 19
4 + 3 = 7
```

Using a `Scanner` in Multiple Methods

- **If you want to read data from the keyboard in multiple methods, you have three choices**

- **You could declare a new `Scanner` object in each method**
  - It is easy to copy and paste the Scanner declaration over and over
  - This leads to duplicate code (yuck!)

- **You could declare one `Scanner` object at the class level**
  - However, because it is accessed by static methods, the Scanner object must be assigned to static instance
  - A static class member is shared by all instances of the class

- **You could pass the `Scanner` object from `main` to any method that needs to use it**
  - This requires an extra parameter

Example: Re-declaring the Scanner

Example

```
import java.util.*;

public class InputApp {
   public static void main(String[] args) {
     Scanner scanner = new Scanner(System.in);
     System.out.print("What do you want to do (1-add, 2= subtract) ?");

     int command = scanner.nextInt();
     if (command == 1) {
         doAdd();
     }
     else if (command == 2) {
         doSubtract();
     }
     else {
         System.out.printf("%d -- Invalid command!", command);
     }
  }

   public static void doAdd() {
     Scanner scanner = new Scanner(System.in);

     System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double sum = num1 + num2;
     System.out.printf("%f + %f = %f", num1, num2, sum);
   }

   public static void doSubtract() {
     Scanner scanner = new Scanner(System.in);

     System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double difference = num1 - num2;
     System.out.printf("%f + %f = %f", num1, num2, difference);
   }
}
```

TRACE OF TERMINAL WINDOW SESSION

```
What do you want to do (1-add, 2= subtract) ? 1

Enter 1st number: 25.25
Enter 2nd number: 5.25
25.25 + 5.25 = 30.500000
```

Example: Passing the Scanner

Example

```
import java.util.*;

public class InputApp {

   public static void main(String[] args) {
     Scanner scanner = new Scanner(System.in);
     System.out.print("What do you want to do (1-add, 2= subtract) ?");

     int command = scanner.nextInt();
     if (command == 1) {
         doAdd(scanner);
     }
     else if (command == 2) {
         doSubtract(scanner);
     }
     else {
         System.out.printf("%d -- Invalid command!", command);
     }
  }

   public static void doAdd(Scanner scanner) {

     System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double sum = num1 + num2;
     System.out.printf("%f + %f = %f", num1, num2, sum);
   }

   public static void doSubtract(Scanner scanner) {

    System.out.print("Enter 1st number: ");
     double num1 = scanner.nextDouble();

     System.out.print("Enter 2nd number: ");
     double num2 = scanner.nextDouble();

     double difference = num1 - num2;
     System.out.printf("%f - %f = %f", num1, num2, difference);
   }
}
```

TRACE OF TERMINAL WINDOW SESSION

```
What do you want to do (1-add, 2= subtract) ? 1

Enter 1st number: 25.25
Enter 2nd number: 5.25
25.25 + 5.25 = 30.500000
```

Exercises – Calculator with Methods

In these exercises you will practice methods.

Remember to create your projects in the `pluralsight/workbook-1` folder.

EXERCISE 1

Create a Java application named method-practice.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always share your IntelliJ project to GitHub

1. Create a new package `com.pluralsight` and a new file named `Hello.java`
2. Create these methods:
   a. main()
   b. sayHello() that prints "Hello, World!"
   c. sayGoodbye() that prints "Goodbye!"
   d. sayGoodMorning() that prints "Good morning!"
   e. Call each method from main()

3. Create a new file named `Menu.java`
   a. Create main() method
   b. Create a method called displayMenu() that prints:

```
"=== MENU ==="
"1. Coffee - $3.99"
"2. Tea - $2.99"
"3. Cookie - $1.99"
```

   c. Call it twice from main() to see it's reusable

4. Create a new file named `Format.java`
    a. Create a main() method
        a. Create this method: `formatName(String first, String last)`
        b. It should RETURN "Last, First" format
    b. Example: `formatName("Bob", "Smith")` returns "Smith, Bob"
    c. Call it in your main and put it in the () of a `System.out.println()`. **Bonus**: set the return of the method call in a variable and print that!
5. Create a new file named `CompareNumbers.java`
    a. Create a main() method
    b. Create this method: `isEven(int number)`
        a. returns true if even
    c. Create: `isPositive(double number)`
    d. returns true if > 0
    e. Call these within a `sout` to view if your methods work or not. EX. `System.out.println(isEven(5));` should display `false`

1. **Ensure all your changes are committed and pushed to GitHub**
2. **Send your repository URL to your Instructor**

In this exercise you will refactor the code from your previous payroll calculator exercise. Refactoring is the process of updating existing code and breaking it into smaller methods to make it more modular and easier to maintain.

EXERCISE 2

Before you start making changes, make a plan to consider how you will break up your code. Use your notebook to diagram your plan. Talk to the people in your breakout room to discuss ideas about how you might do it.

Step 1

Code your changes

When you finish, we will get together as a class and examine some of the different approaches people used.

Step 2

1. Ensure all your changes are committed and pushed to GitHub
2. Send your repository URL to your Instructor

Section 4–3

The `switch` Statement

`switch` Statement

- **The `switch` statement evaluates an expression and executes the code in a matching `case` block**
  - If there is no match, the `default` block executes
  - Typically switch statements are used to check many possibilities for one variable

Syntax

```
switch(expression) {
  case value1 -> {
    // code block
  }
  case value2 -> {
    // code block
  }
  default -> {
    // code block
  }
}
```

`switch` Example

- **Switch statements can get pretty long if the expression has many unique values**
  - It's still easier to read than the equivalent if/else construct
  - They will also allow for easy setting of many possibilities to one variable

Example

```
int dayNumber = 3;

String description = switch (dayNumber) {
  case 0 -> "Sunday";
  case 1 -> "Monday";
  case 2 -> "Tuesday";
  case 3 -> "Wednesday";
  case 4 -> "Thursday";
  case 5 -> "Friday";
  case 6 -> "Saturday";
  default -> "Invalid day";
};
```

- **Compared to an if statement, we would have to write a lot more code!**

```
int dayNumber = 3;
String description;

if (dayNumber == 0) {
     description = "Sunday";
} else if (dayNumber == 1) {
     description = "Monday";
} else if (dayNumber == 2) {
     description = "Tuesday";
} else if (dayNumber == 3) {
     description = "Wednesday";
} //and so on…
```

The break statement

- **If a `case` block doesn't end with a `break`, then code falls into the next block**
  - This is intentional; it can make your switch statement more compact when several values lead to the same behavior

Example

```
int dayNumber = 3;
String description = "";

switch (dayNumber) {
  case 0:
  case 6:
    description = "Weekend";
    break;
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    description = "Weekday";
    break;
}
```

- **In Java version 14's syntax, the break; is now not needed**

Example

```
int dayNumber = 3;

String description = switch (dayNumber) {
  case 0, 6 -> "Weekend";
  case 1, 2, 3, 4, 5 -> "Weekday";
  default -> "Invalid day";
};
```

Exercises – Rental Car Company

In this exercise you will create a CLI application for a rental car company. You will prompt the user to answer questions about their selected options and then calculate the cost of the rental.

Remember to do your work in the `workbook-1` directory.

Remember: When creating a new Java project, create a new git repository and commit your changes often! Don't forget to push to github.com. Always share your IntelliJ project to GitHub

You are not required to use only `switch` statements in this exercise. You can choose to use `if/else` or `switch` statements as you deem appropriate.

EXERCISE 1

Step 1

IMPORTANT: DO NOT just start coding - take time to plan the flow of your application in your notebook.

Read the requirements below and consider how you would calculate the cost if you did not have a computer. Practice several scenarios (with different options) in your notebook before you create your project in IntelliJ.

Step 2

Create a Java application named `rental-car-calculator` that estimates the cost of reserving a rental car. Prompt the user for the following information:

a. pickup date (store as a string)

b. number of days for the rental

c. whether they want an electronic toll tag at $3.95/day (yes/no)

d. whether they want a GPS at $2.95/day (yes/no)

e. whether they want roadside assistance at $3.95/day (yes/no)

f. their current age

Calculate and display:

a. basic car rental

b. options cost

c. underage driver surcharge

d. total cost

The basic car rental is $29.99 per day. There is a 30% surcharge on the basic car rental for drivers under 25. All taxes have already been incorporated into the fees shown.

Step 3

1. Ensure all your changes are committed and pushed to GitHub

2. Send your repository URL to your Instructor

Conditional Operator

- **The conditional operator (? :) in Java is like a shorthand version of `if/else`**
  - A similar operator is found in all C-family languages

Syntax

```
variable = (condition) ? expressionTrue : expressionFalse;
```

Example

```
String name = "Ezra";
int age = 17;

double price = (age < 18) ? 22.50 : 25.00;

System.out.println("Price of admission: $" + price);
//this prints: Price of admission: $22.50
```

∗ Compare this with an if and else statement:

```
String name = "Ezra";
int age = 17;

double price;

if (age < 18) {
     price = 22.50;
} else {
     price = 25.00;
}

System.out.println("Price of admission: $" + price);
//this prints: Price of admission: $22.50
```

Module 5

Continued Learning

Section 5–1

Debugging Project

Debugging Project

- **Working with existing projects will be most of your work in your career.**

- **Debugging projects you are unfamiliar with will help you become a more robust developer**
  - Download the Pizza Order System project from our week 1 folder in the **Bonus Repository**
  - The project includes `PizzaOrderSystem.java` (the main file with bugs) and supporting documentation
  - Extract the files to a `debugging-projects` directory you create in the `~/pluralsight` directory

- **Open IntelliJ IDEA and create a new Java project**
  - Copy the `PizzaOrderSystem.java` file into your project's `src` folder
  - Review both the **Student Guide** and **Bug Tickets** documents before starting

- **Start by reading through all 8 bug tickets to understand what needs fixing**

- **Use the Progress Tracker in the Student Guide to check off bugs as you fix them**

- **Follow the Debugging Strategy section: fix compilation errors first, then test each feature**

Section 5–2

Practice Repository

Practice Problems

- **We have a repository with additional practice problems and projects**
- **Your Instructor will inform you which exercises and projects you will work on within this repository**
- **If the files contain -optional, you are more than welcome to work on these as you please**
- **`https://github.com/Pluralsight-ILT/YUU-Java-Academy-Bonus`**